System Architecture — Big Data Ingestion + Credit-Regulated API

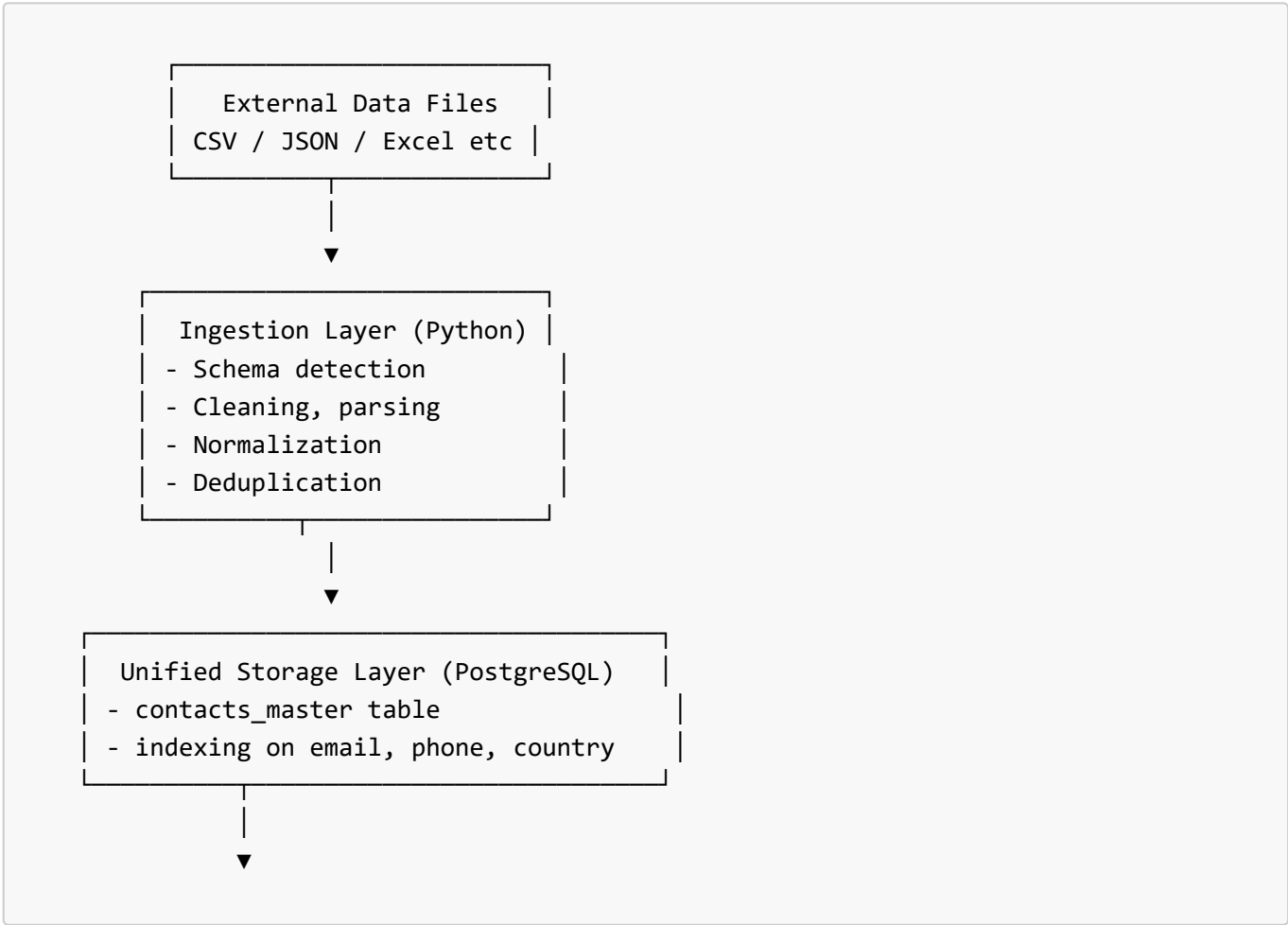
1. Overview

This architecture supports:

- Ingestion of **70M–700M+** records from multiple formats
- Schema normalization and deduplication
- Scalable storage using PostgreSQL
- Fast credit-regulated API using FastAPI
- API logging, authentication, pagination, and filtering
- Support for future scale-up to distributed systems

---

2. High-Level Architecture Diagram



api_logs table │ │ - users & credits tables │
└──┘

---

3. Ingestion Architecture

Responsibilities:

- ✔ Load multi-format data (CSV, Excel, TSV, JSON, JSONL)
- ✔ Detect & normalize schemas
- ✔ Clean malformed rows
- ✔ Convert to unified master schema
- ✔ Remove duplicates `(email, phone)`
- ✔ Batch insert into PostgreSQL using COPY

Tools Used:

- Python
- Pandas
- Custom normalization functions
- SQLAlchemy
- PostgreSQL COPY command for fast batching

---

4. Storage Architecture

Database: PostgreSQL

Chosen because it supports large datasets, indexing, partitioning, and fast queries.

Tables:

- `contacts_master`
- `users`
- `credits`
- `api_logs`

Recommended Indexes:

```
CREATE INDEX idx_country ON contacts_master(country);
CREATE INDEX idx_email ON contacts_master(email);
CREATE INDEX idx_phone ON contacts_master(phone);
Scaling Strategy:
Add read replicas when traffic grows

Partition contacts_master by country or ranges

Consider Citus/PostgreSQL sharding for 500M+ rows

5. API Architecture
```

```
Framework: FastAPI

Features:
API Key authentication

Credit usage deduction

Pagination

Dynamic filtering (country, email, phone, company)

Detailed response metadata

SQLAlchemy ORM + connection pooling

API Flow:
Validate API Key

Check credits

Query dataset

Deduct credits

Log the call

Return paginated result

6. Logging Architecture
Logs stored in api_logs table:

Each record includes:

user_id

endpoint

query_params

credits_used

response_time_ms

called_at

Used for:

Billing

Analytics

Fraud detection

Performance monitoring
```

```
7. Scalability Plan
Short-term:
Add indexes

Use connection pooling

Use Redis for frequent queries (optional)

Medium-term:
PostgreSQL partitioning

Task queues for ingestion (Celery / Redis)

Long-term (700M+ records):
Move to Citus, ClickHouse, or BigQuery

Use S3-based data lake

Spark/Flink for ingestion at scale

8. Deployment Architecture
Development:
FastAPI

PostgreSQL (local)

Docker optional

Production:
Docker Compose OR Kubernetes

Nginx reverse proxy

SSL termination

Load balancer

Remote PostgreSQL instance

9. Summary
This architecture ensures:

✔ Scalable ingestion
✔ Clean unified schema
✔ Fast search API
✔ Credit-based usage control
✔ Full logging and auditing
✔ Expandable to 1 billion+ records


---

If you want, I can resend **all other docs again** too.
```

```
Just tell me:
**“send next document”**
```